

IncidentFlow

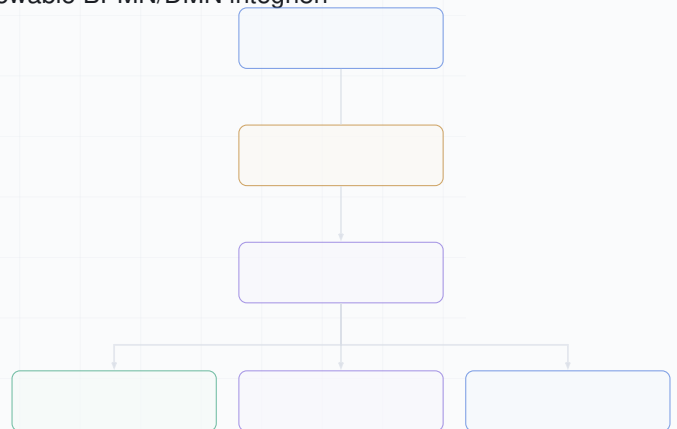
V3

Hyper-automatisation avec n8n

Passer d'une orchestration manuelle à un système « Zero-Touch » : Spring Boot et Flowable pilotent le processus BPMN, tandis que n8n exécute les intégrations via Webhooks, Service Tasks et APIs.

IncidentFlow → V3 → Hyper-automatisation → n8n → BPMN → Zero-Touch

■ Spring Boot , primPostgreSQL primReact warnFlowable BPMN/DMN integ n8n
integWebhooks & APIs succKeycloak



NETMAR
Your Trusted Partner

Technical Architecture & Automation

Rapport d'architecture détaillé | 2 octobre 2026

Réalisé par : Anas Haddou

École Marocaine d'Ingénierie



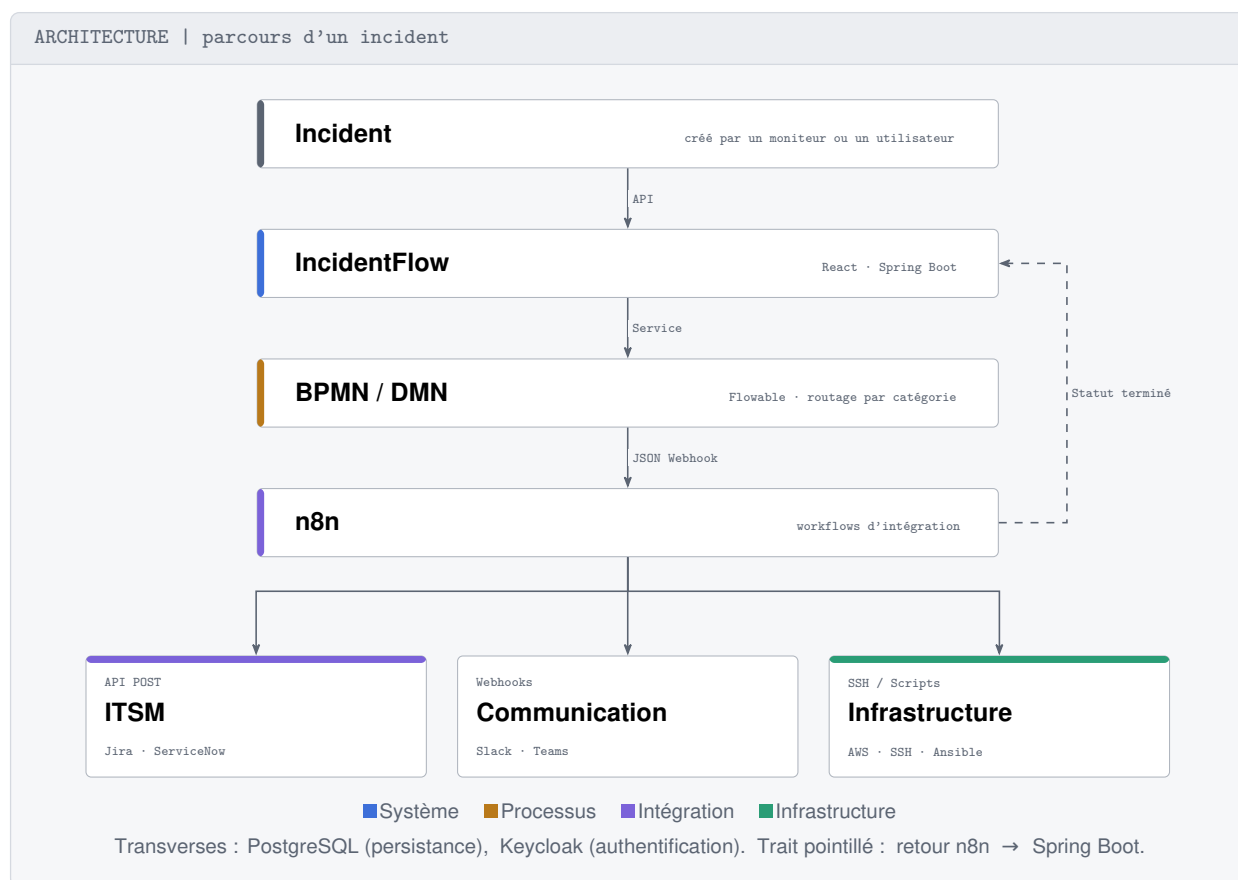
المدرسة المغربية للهندسة
ECOLE MAROCAINE D'INGENIERIE

■ Résumé

Ce document présente la vision architecturale pour la version 3 du système IncidentFlow. Après avoir mis en place une orchestration métier robuste via Flowable BPMN et une sécurité centralisée via Keycloak (V2), la V3 se concentre sur l'automatisation des tâches (Zero-Touch) et l'intégration avec des systèmes tiers via l'outil d'automatisation n8n. Ce rapport détaille les scénarios d'usage, les modifications applicatives (Front/Back/BDD) et les ressources techniques nécessaires.

Architecture globale V3

Spring Boot et Flowable gardent la logique métier ; n8n porte toutes les intégrations externes. La figure suit le parcours d'un incident, de sa création à son exécution.



01 | Scénarios d'usage : Zero-Touch vs fallback humain

L'automatisation intelligente ne remplace pas l'humain, elle agit comme un filtre puissant pour ne lui laisser que les tâches complexes ou physiques.

Contexte	Problème	Solution	Résultat
Un outil de supervision (ex : Zabbix) crée l'incident via l'API Spring Boot.	Tous les incidents ne sont pas automatisables : action physique, validation managériale, API prestataire en échec.	Le moteur DMN route vers une <code>ServiceTask</code> n8n ou vers une <code>UserTask</code> .	Cas technique : résolution sans intervention humaine.

1.1 Scénario A : 100 % automatisé (Zero-Touch)

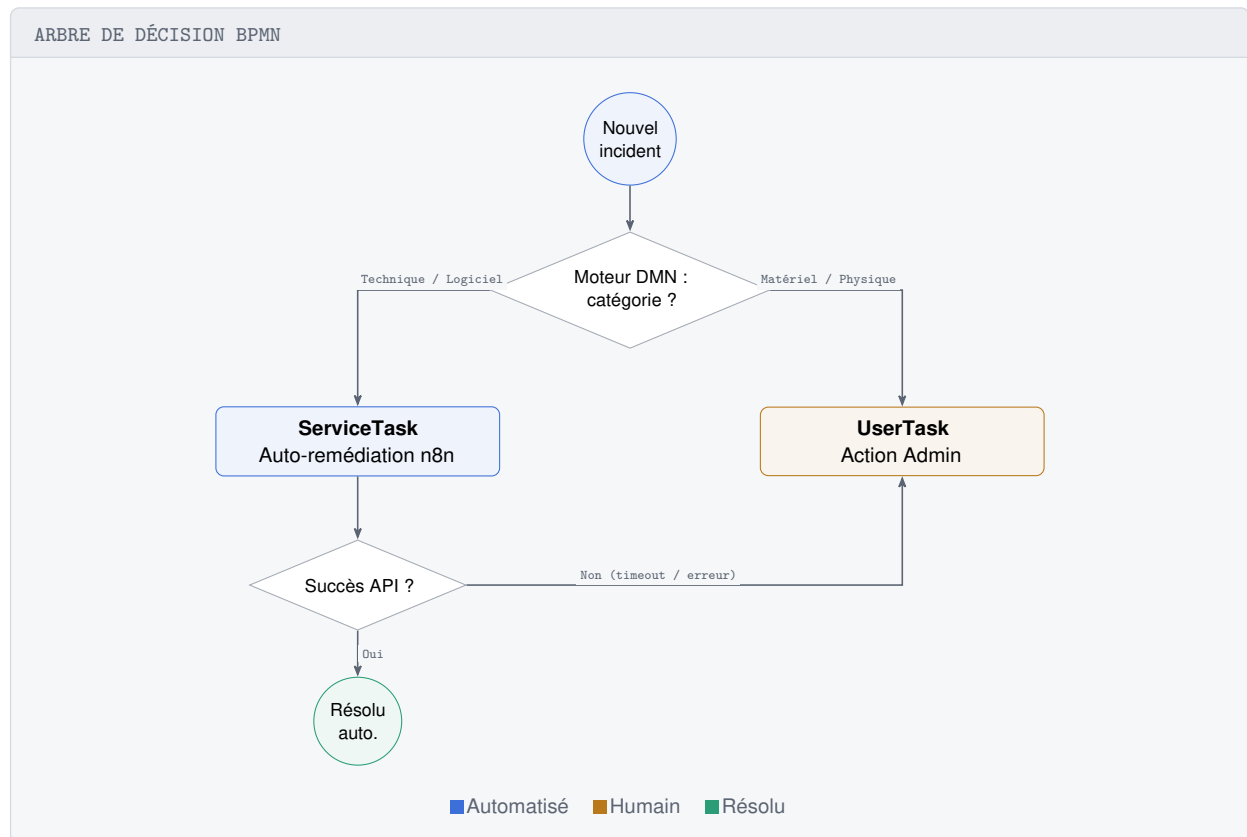
Cas d'usage : un serveur web devient soudainement inaccessible.

- L'outil de supervision (ex : Zabbix) crée l'incident via notre API Spring Boot.
- Le moteur de règles DMN (Flowable) identifie un problème technique critique.
- Une `ServiceTask` transmet le payload à **n8n**.
- n8n exécute un script distant (SSH / Ansible) pour redémarrer le service, et notifie le canal Slack de l'équipe d'astreinte.
- Si le serveur répond à nouveau, n8n clôture l'incident dans notre base de données.
- *Temps de résolution : 2 secondes, sans aucune intervention humaine.*

1.2 Scénario B : l'exception humaine (fallback)

Cas d'usage : une imprimante est cassée, ou un script d'auto-remédiation a échoué (timeout API).

- Le système détecte qu'une action physique ou une validation managériale est requise.
- Le processus BPMN quitte la boucle d'automatisation.
- Il génère automatiquement une `UserTask` assignée au bon technicien (via Keycloak) pour procéder à une réparation manuelle.



02 | Chantiers techniques : code et base de données

Pour supporter cette V3, plusieurs évolutions du code source sont planifiées.

Contexte	Problème	Solution
La V3 ajoute des actions exécutées par des machines sur des tickets existants.	Tracer l'historique des scripts n8n, éviter les conflits humain / machine, sécuriser les échanges.	Nouvelles colonnes, <code>WebhookController</code> , API Keys, dashboard et mode lecture seule.

■ Base de données (PostgreSQL)

Le schéma de la table `Incident` sera enrichi avec de nouvelles métadonnées :

- `is_automated` (boolean) : flag indiquant si le ticket est géré par la machine.
- `external_ticket_id` (varchar) : référence du ticket du sous-traitant (ex : JIRA-901).
- `remediation_logs` (text) : historique des scripts n8n exécutés.
- Nouvelle table `N8N_WEBHOOK_EVENT` : historique de chaque action prise par l'IA ou les scripts externes.

■ Backend (Spring Boot & Flowable)

- **Moteur DMN** : activation du moteur de règles Flowable pour déduire la criticité.
- **Nouveaux Controllers REST** : développement d'un `WebhookController.java` protégé par API Keys pour recevoir les retours asynchrones de n8n, sans passer par l'interface de login React.

■ Frontend (React)

- **Nouveau dashboard de supervision** : une vue dédiée affichant en temps réel les incidents en cours d'auto-remédiation.
- **Verrouillage** : désactivation du bouton « Traiter » sur l'UI si `is_automated=true`, afin d'éviter qu'un humain n'interfère avec le script.

MISE À JOUR DE LA BASE DE DONNÉES



03 | Écosystème, ressources et intégrations API

L'intégration de la V3 nécessitera la mise en place de nouvelles ressources dans notre architecture Docker.

Contexte

n8n devient la couche d'intégration entre IncidentFlow et le monde extérieur.

Problème

Répliquer les incidents chez le sous-traitant, alerter l'astreinte, agir sur les serveurs.

Solution

Un conteneur n8n, une file Redis et des connecteurs ITSM, chat et infrastructure.

■ Infrastructures et ressources

Ajout du conteneur `n8nio/n8n` au fichier `docker-compose.yml`, ainsi qu'une base Redis supplémentaire pour gérer la file d'attente des webhooks.

3.1 Intégrations prévues

Famille	Outils	Usage
Systèmes ITSM externes (Société X)	Jira Service Management, ServiceNow	n8n utilise les API REST de ces outils pour répliquer nos incidents dans le système d'information du prestataire et suivre leur résolution en temps réel.
Communication et alertes	Slack API, Microsoft Teams Webhooks, serveur SMTP (email)	Notifications immédiates des administrateurs, avec « boutons d'action » directement dans le chat Slack pour approuver un redémarrage serveur.
Infrastructure et cloud	AWS EC2 API, SSH Node (n8n), Ansible	Redémarrage automatique des instances cloud, purge de cache ou de mémoire sans intervention humaine.

04 | Conclusion et avantages stratégiques

En isolant toutes les communications API externes (Jira, Slack, fournisseurs) dans n8n, nous protégeons le code source Spring Boot de l'inflation technologique. La séparation des responsabilités entre Java (métier) et n8n (intégrations) apporte trois gains directs.

Découplage extrême

Le code Java reste pur : pas de SDK officiels Slack ou Jira à coder ni à maintenir dans Spring Boot. L'intégration est déléguée visuellement à n8n.

Réduction du MTTR

Le temps moyen de résolution (MTTR) est drastiquement réduit : les incidents critiques notifient les bons experts techniques et leurs systèmes, nuit et jour, instantanément.

Évolutivité facile

Si la « Société X » change d'API, il n'est plus nécessaire de recompiler le backend Spring Boot : il suffit de modifier le flux visuellement, en glisser-déposer, sur n8n.

La version 3 transforme un simple outil de « ticketing » en une **plateforme d'hyper-automatisation ITSM** robuste, et offre à l'entreprise une capacité d'intégration visuelle immédiate pour de futurs partenaires.